

Lab 3: Using Classification to Build a Marketing Target Model

Data 622

Umais Siddiqui

2026-02-14

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (classification_report, roc_curve, auc,
                             precision_score, recall_score, f1_score,
                             accuracy_score, confusion_matrix)

import warnings
warnings.filterwarnings('ignore')

sns.set_style("whitegrid")
pd.set_option('display.max_columns', 25)
pd.set_option('display.width', 120)
```

```
df = pd.read_csv("DataSets/bank_marketing_dataset.csv")
print(f"Dataset shape: {df.shape}")
df.head()
```

Dataset shape: (45211, 24)

	sno	age	job	marital	education	default	balance	housing	loan	contact	day	dura
0	0	58	management	married	tertiary	no	2143	yes	no	unknown	5	261
1	1	44	technician	single	secondary	no	29	yes	no	unknown	5	151
2	2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	76
3	3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	92
4	4	33	unknown	single	unknown	no	1	no	no	unknown	5	198

Problem 1: What Can Machine Learning Do?

(a) How ML Can Help the Marketing Team

A classification model trained on historical campaign data can help the marketing team in several concrete ways. First, and most directly, it can **score and rank prospective clients** by their predicted probability of subscribing to a term deposit, allowing the team to prioritize high-probability leads and avoid wasting calls on unlikely converters. This targeted approach increases the hit rate (fraction of successful calls) and reduces the cost per acquisition. If the model can identify the top 10% of clients who are, say, $3\times$ more likely to subscribe, the team could achieve the same number of conversions with far fewer calls — or achieve many more conversions with the same budget.

Second, the model can provide **interpretable insights** into which client characteristics and campaign strategies drive success. For example, if the model reveals that cellular contact is far more effective than telephone, or that clients who previously subscribed are the easiest to re-convert, the team can redesign their outreach strategy accordingly. The model can also help identify the best timing for campaigns by revealing how macroeconomic conditions (like the Euribor interest rate) affect client receptivity — the team might accelerate campaigns when conditions are favorable and scale back when the environment is hostile to term deposit sales.

(b) The Kaggle Notebook Flaw

The most upvoted Kaggle notebook includes **duration** (the length of the phone call in seconds) as a predictor variable. This is a fatal flaw because **duration** is only known *after* the call has been made — it cannot be used to decide *who* to call. As the UCI documentation explicitly warns, duration should be discarded for any realistic predictive model. Including it inflates model performance dramatically (since longer calls strongly predict success), but the model is completely useless in practice because you would need to already have called the client to know how long the call lasted.

Problem 2: EDA and Baseline Model

(a) EDA and Temporal Patterns

```
# Basic data overview
print("Column dtypes:")
print(df.dtypes)
print(f"\nTarget distribution:")
print(df['y'].value_counts())
print(f"\nTarget success rate: {(df['y']=='yes').mean():.4f} ({(df['y']=='yes').sum()} / {len(df)})")
```

Column dtypes:

sno	int64
age	int64
job	str
marital	str
education	str
default	str
balance	int64
housing	str
loan	str
contact	str
day	int64
duration	int64
campaign	int64
pdays	int64
previous	int64
poutcome	str
month_numeric	int64
year	int64
euribor3m	float64
emp.var.rate	float64
nr.employed	float64
cons.price.idx	float64
cons.conf.idx	float64
y	str

dtype: object

Target distribution:

y	
no	39922

```
yes      5289
Name: count, dtype: int64
```

Target success rate: 0.1170 (5289 / 45211)

```
print("Missing values:")
print(df.isnull().sum()[df.isnull().sum() > 0])
if df.isnull().sum().sum() == 0:
    print("No missing values in any column.")
print(f"\nBasic statistics (numeric):")
df.describe().round(2)
```

Missing values:
Series([], dtype: int64)
No missing values in any column.

Basic statistics (numeric):

	sno	age	balance	day	duration	campaign	pdays	previous	month_numeric
count	45211.00	45211.00	45211.00	45211.00	45211.00	45211.00	45211.00	45211.00	45211.00
mean	22605.00	40.94	1362.27	15.81	258.16	2.76	40.20	0.58	6.14
std	13051.44	10.62	3044.77	8.32	257.53	3.10	100.13	2.30	2.41
min	0.00	18.00	-8019.00	1.00	0.00	1.00	-1.00	0.00	1.00
25%	11302.50	33.00	72.00	8.00	103.00	1.00	-1.00	0.00	5.00
50%	22605.00	39.00	448.00	16.00	180.00	2.00	-1.00	0.00	6.00
75%	33907.50	48.00	1428.00	21.00	319.00	3.00	-1.00	0.00	8.00
max	45210.00	95.00	102127.00	31.00	4918.00	63.00	871.00	275.00	12.00

```
# Create target binary
df['y_bin'] = (df['y'] == 'yes').astype(int)
df['time_order'] = df['year'] * 100 + df['month_numeric']

# Temporal success rate
temporal = df.groupby('time_order').agg(
    n=('y', 'size'),
    success_rate=('y_bin', 'mean'),
    euribor3m=('euribor3m', 'first'),
    emp_var_rate=('emp.var.rate', 'first')
).reset_index()
```

```

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Success rate over time
axes[0, 0].bar(range(len(temporal)), temporal['success_rate'], color='steelblue')
axes[0, 0].set_xticks(range(len(temporal)))
axes[0, 0].set_xticklabels(temporal['time_order'].astype(str), rotation=45, ha='right', font)
axes[0, 0].set_ylabel('Success Rate')
axes[0, 0].set_title('Campaign Success Rate Over Time')
axes[0, 0].axhline(y=df['y_bin'].mean(), color='red', linestyle='--', alpha=0.7, label='Over')
axes[0, 0].legend()

# Volume over time
axes[0, 1].bar(range(len(temporal)), temporal['n'], color='coral')
axes[0, 1].set_xticks(range(len(temporal)))
axes[0, 1].set_xticklabels(temporal['time_order'].astype(str), rotation=45, ha='right', font)
axes[0, 1].set_ylabel('Number of Calls')
axes[0, 1].set_title('Campaign Volume Over Time')

# Euribor3m over time
axes[1, 0].plot(range(len(temporal)), temporal['euribor3m'], 'o-', color='darkgreen')
axes[1, 0].set_xticks(range(len(temporal)))
axes[1, 0].set_xticklabels(temporal['time_order'].astype(str), rotation=45, ha='right', font)
axes[1, 0].set_ylabel('Euribor 3-month rate (%)')
axes[1, 0].set_title('Euribor 3-Month Rate Over Time')

# Success rate vs euribor
axes[1, 1].scatter(temporal['euribor3m'], temporal['success_rate'], s=temporal['n']/10,
                    alpha=0.7, color='steelblue', edgecolors='black')
axes[1, 1].set_xlabel('Euribor 3-month rate (%)')
axes[1, 1].set_ylabel('Success Rate')
axes[1, 1].set_title('Success Rate vs Euribor3m (size = volume)')

plt.tight_layout()
plt.show()

```

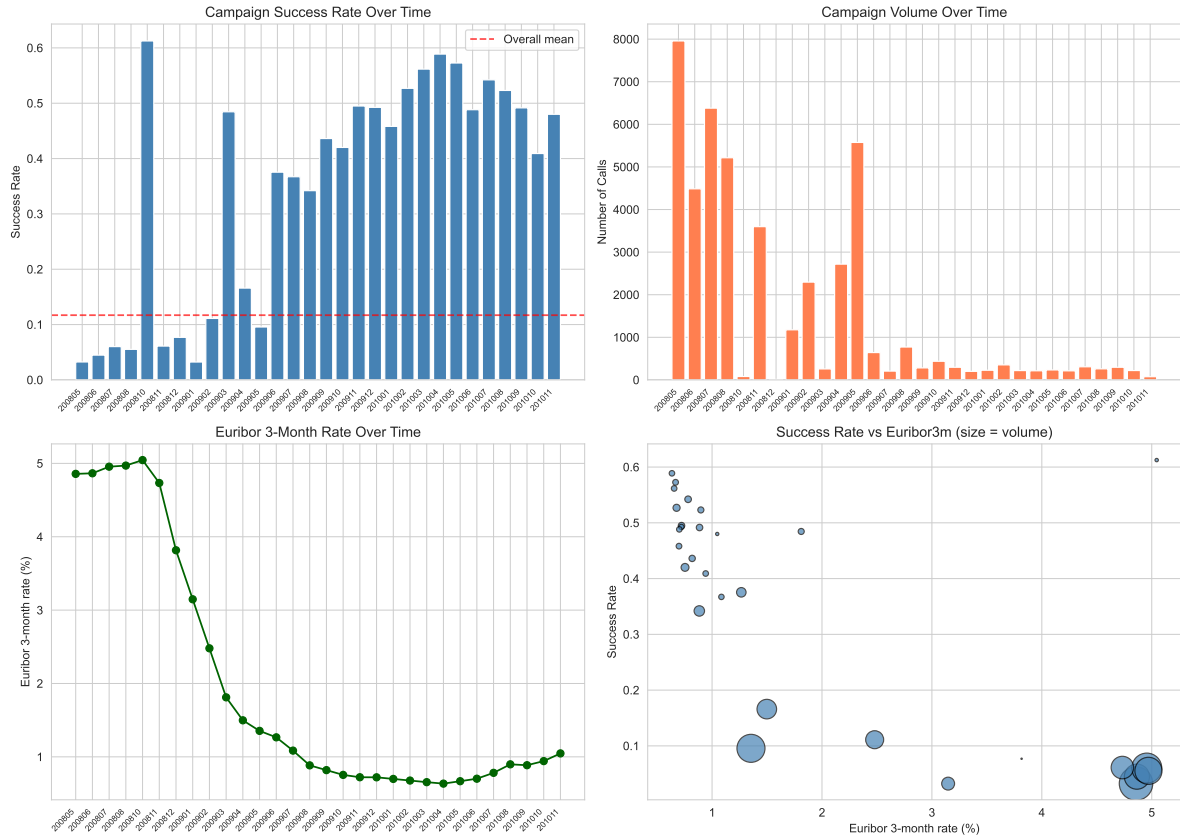


Figure 1: Temporal patterns in campaign success rate and volume

```
print("Success rate by year:")
print(df.groupby('year').agg(
    n=('y', 'size'),
    success_rate=('y_bin', 'mean'),
    mean_euribor=('euribor3m', 'mean'),
    mean_emp_var_rate=('emp.var.rate', 'mean')
).round(4))
```

Success rate by year:

	n	success_rate	mean_euribor	mean_emp_var_rate
year				
2008	27729	0.0505	4.8861	1.1142
2009	14862	0.1706	1.6231	-2.0457
2010	2620	0.5160	0.7657	-1.6120

```

fig, axes = plt.subplots(2, 3, figsize=(16, 10))

# Contact method
ct = pd.crosstab(df['contact'], df['y'], normalize='index')
ct.plot(kind='bar', ax=axes[0, 0], color=['#d9534f', '#5cb85c'])
axes[0, 0].set_title('Success Rate by Contact Method')
axes[0, 0].set_ylabel('Proportion')
axes[0, 0].tick_params(axis='x', rotation=0)

# Previous outcome
ct2 = pd.crosstab(df['poutcome'], df['y'], normalize='index')
ct2.plot(kind='bar', ax=axes[0, 1], color=['#d9534f', '#5cb85c'])
axes[0, 1].set_title('Success Rate by Previous Outcome')
axes[0, 1].set_ylabel('Proportion')
axes[0, 1].tick_params(axis='x', rotation=0)

# Housing loan
ct3 = pd.crosstab(df['housing'], df['y'], normalize='index')
ct3.plot(kind='bar', ax=axes[0, 2], color=['#d9534f', '#5cb85c'])
axes[0, 2].set_title('Success Rate by Housing Loan')
axes[0, 2].set_ylabel('Proportion')
axes[0, 2].tick_params(axis='x', rotation=0)

# Job
job_rates = df.groupby('job')['y_bin'].mean().sort_values(ascending=False)
job_rates.plot(kind='bar', ax=axes[1, 0], color='steelblue')
axes[1, 0].set_title('Success Rate by Job')
axes[1, 0].set_ylabel('Success Rate')
axes[1, 0].tick_params(axis='x', rotation=45)

# Age distribution by outcome
df[df['y']=='no']['age'].hist(ax=axes[1, 1], bins=30, alpha=0.5, color='#d9534f', label='No')
df[df['y']=='yes']['age'].hist(ax=axes[1, 1], bins=30, alpha=0.5, color='#5cb85c', label='Yes')
axes[1, 1].set_title('Age Distribution by Outcome')
axes[1, 1].set_xlabel('Age')
axes[1, 1].legend()

# Balance distribution by outcome
df[df['y']=='no']['balance'].clip(-2000, 10000).hist(ax=axes[1, 2], bins=30, alpha=0.5,
    color='#d9534f', label='No', density=True)
df[df['y']=='yes']['balance'].clip(-2000, 10000).hist(ax=axes[1, 2], bins=30, alpha=0.5,
    color='#5cb85c', label='Yes', density=True)

```

```

axes[1, 2].set_title('Balance Distribution by Outcome')
axes[1, 2].set_xlabel('Balance (€)')
axes[1, 2].legend()

plt.tight_layout()
plt.show()

```

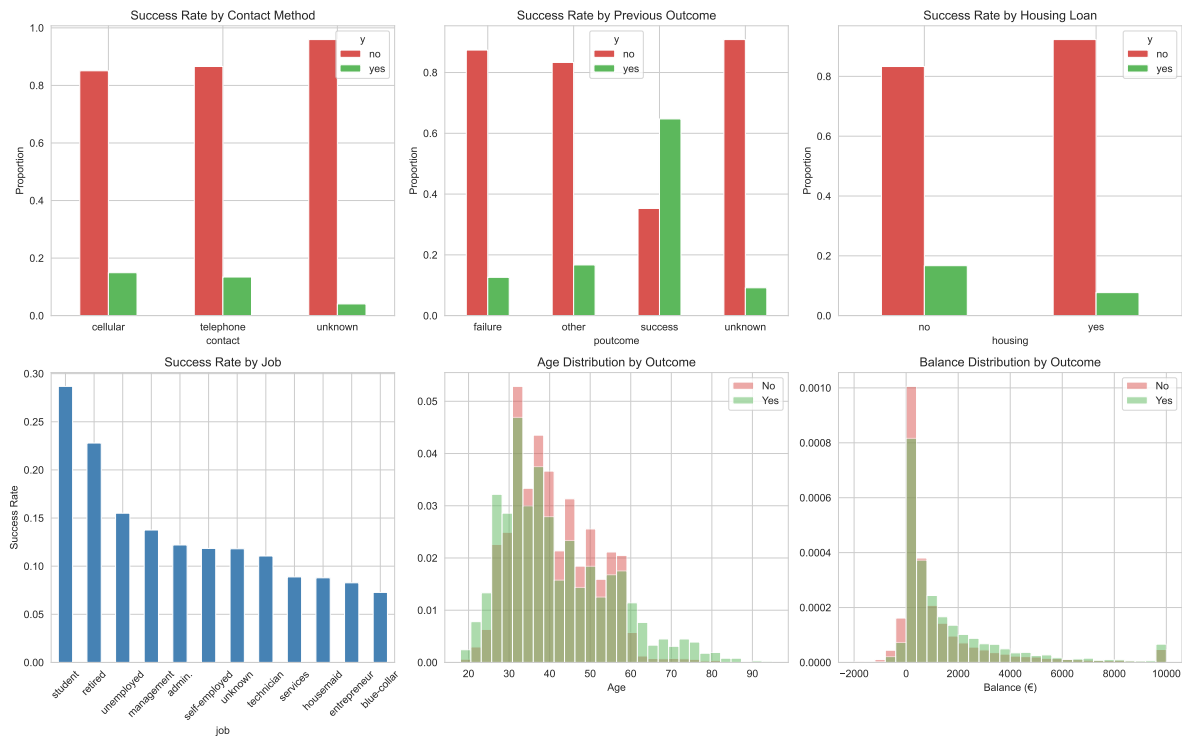


Figure 2: Key predictor relationships with the target variable

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Contact method over time
ct_time = pd.crosstab(df['time_order'], df['contact'], normalize='index')
ct_time.plot(kind='bar', stacked=True, ax=axes[0], width=0.8)
axes[0].set_title('Contact Method Distribution Over Time')
axes[0].set_ylabel('Proportion')
axes[0].tick_params(axis='x', rotation=45)
axes[0].legend(loc='upper right')

# Job category shift - retired + student share over time

```

```

df['is_high_success_job'] = df['job'].isin(['retired', 'student']).astype(int)
job_share = df.groupby('time_order')['is_high_success_job'].mean()
axes[1].plot(range(len(job_share)), job_share.values, 'o-', color='purple')
axes[1].set_xticks(range(len(job_share)))
axes[1].set_xticklabels(job_share.index.astype(str), rotation=45, ha='right', fontsize=7)
axes[1].set_ylabel('Share of Retired + Student')
axes[1].set_title('Share of High-Success Job Categories Over Time')

plt.tight_layout()
plt.show()

```

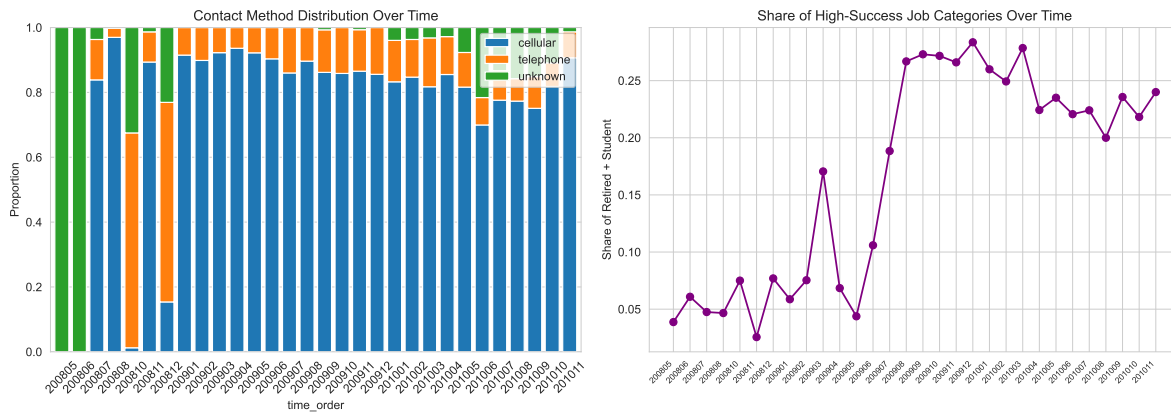


Figure 3: Contact method and population composition over time

Summary of key findings from EDA:

The most striking pattern in this dataset is the **dramatic temporal variation** in campaign success. The success rate rises from roughly 5% in 2008 to 17% in 2009 to over 50% in 2010. This variation is primarily driven by **macroeconomic conditions**, specifically the Euribor 3-month interest rate and the employment variation rate.

The **Euribor 3-month rate** — the interbank lending rate in the Eurozone — declined sharply from ~5% in mid-2008 to below 1% by late 2009, driven by the ECB's aggressive rate cuts during the 2008–2009 financial crisis. When market interest rates are low, bank term deposits become relatively more attractive (they offer fixed returns in a low-rate environment) and the opportunity cost of locking up money is lower. The correlation between the euribor3m rate and campaign success is strongly negative: as the rate fell, success rates soared.

The **employment variation rate** (`emp.var.rate`) captures labor market conditions. It turned sharply negative during the crisis (from +1.4 to -3.4), reflecting rising unemployment. Counterintuitively, worse employment conditions are associated with *higher* term deposit uptake

— likely because economic uncertainty makes safe, fixed-return savings products more appealing to risk-averse consumers.

Beyond macroeconomics, the **composition of the target population changed** over time. In 2010, the share of retired people and students called was notably higher — these are the highest-success-rate job categories. The **contact method** also shifted: the first two months (May–June 2008) used only “unknown” contact, while later periods predominantly used cellular.

Other important predictors include: **poutcome** (clients who previously succeeded are ~65% likely to subscribe again), **housing** (those without housing loans subscribe at higher rates), **contact** (cellular outperforms telephone), and **education** (tertiary-educated clients subscribe more).

```
# Encode target
df['y_bin'] = (df['y'] == 'yes').astype(int)

# Create useful derived variables
df['contacted_before'] = (df['pdays'] != -1).astype(int)
df['prev_success'] = (df['poutcome'] == 'success').astype(int)

# Train-test split: stratify by y to preserve class balance
# We do NOT stratify by epoch/time because the assignment asks us to evaluate
# temporal variation, and stratifying by time would undermine that analysis.
# Stratifying by y ensures both sets have the ~11.7% positive rate.

features_to_drop = ['y', 'y_bin', 'sno', 'duration', # duration is leakage
                    'is_high_success_job'] # temp variable from EDA

X = df.drop(columns=[c for c in features_to_drop if c in df.columns])
y = df['y_bin']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set: {X_train.shape[0]} observations")
print(f"Testing set: {X_test.shape[0]} observations")
print(f"Train success rate: {y_train.mean():.4f}")
print(f"Test success rate: {y_test.mean():.4f}")
```

```
Training set: 36168 observations
Testing set: 9043 observations
Train success rate: 0.1170
```

Test success rate: 0.1170

Stratification choice: I stratify by y (the target variable) rather than by time. The target is imbalanced (~11.7% positive), so stratification ensures both train and test sets preserve this ratio. I do not stratify by time because the assignment specifically asks us to examine how model performance varies across temporal epochs — a random (non-temporal) split serves this purpose, as each epoch will be represented in both train and test sets.

```
# Define epochs based on euribor3m - the primary driver of temporal variation
# Three epochs reflecting distinct macroeconomic regimes:
#   High (>3.5%): Pre-crisis / early crisis (2008), euribor still high
#   Medium (1.5-3.5%): Crisis transition (early-mid 2009), rates falling rapidly
#   Low (<1.5%): Post-crisis (late 2009 - 2010), rates at historic lows

epoch_bins = [0, 1.5, 3.5, 5.5]
epoch_labels = ['Low (<1.5%)', 'Medium (1.5-3.5%)', 'High (>3.5%)']

test_epochs = pd.cut(X_test['euribor3m'], bins=epoch_bins, labels=epoch_labels)

print("Test set epoch distribution:")
print(test_epochs.value_counts().sort_index())
print("\nTest set success rate by epoch:")
for ep in epoch_labels:
    mask = test_epochs == ep
    print(f" {ep}: {y_test[mask].mean():.4f} (n={mask.sum()})")
```

Test set epoch distribution:

euribor3m	
Low (<1.5%)	2679
Medium (1.5-3.5%)	777
High (>3.5%)	5587
Name: count, dtype: int64	

Test set success rate by epoch:

Low (<1.5%):	0.2628 (n=2679)
Medium (1.5-3.5%):	0.1081 (n=777)
High (>3.5%):	0.0483 (n=5587)

(b) Baseline Model and Data Drift

Why temporal variation causes metric mismatch: If we train and evaluate a model on a random split of the entire dataset, the overall performance metrics blend together periods

with very different success rates and different predictor distributions. In 2008, only 5% of calls succeeded (high euribor, pre-crisis); in 2010, over 50% succeeded (low euribor, post-crisis). A model evaluated on the pooled test set might show decent accuracy — but this average hides the fact that the model may perform brilliantly in one epoch and terribly in another. In practice, the marketing team uses the model in real time, where they face one macroeconomic regime at a time. If the current conditions resemble 2008, the pooled metrics will be misleadingly optimistic. Conversely, if conditions resemble 2010, the pooled metrics may understate performance. Evaluating by epoch reveals these regime-specific performance characteristics, which is exactly what the marketing team needs to set realistic expectations.

```
# Define feature sets
cat_features = ['job', 'marital', 'education', 'default', 'housing',
               'loan', 'contact', 'poutcome']
num_features = ['age', 'balance', 'campaign', 'pdays', 'previous',
               'euribor3m', 'emp.var.rate', 'nr.employed',
               'cons.price.idx', 'cons.conf.idx']

features_baseline = num_features + cat_features

# Build sklearn pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore',
                               sparse_output=False), cat_features)
    ]
)

baseline_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))
])

# Fit on full training set
baseline_pipeline.fit(X_train[features_baseline], y_train)
print("Baseline logistic regression fitted.")
```

Baseline logistic regression fitted.

```
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
```

```

results = []

for i, epoch_name in enumerate(epoch_labels):
    mask = test_epochs == epoch_name
    X_ep = X_test[features_baseline][mask]
    y_ep = y_test[mask]

    y_pred = baseline_pipeline.predict(X_ep)
    y_prob = baseline_pipeline.predict_proba(X_ep)[:, 1]

    acc = accuracy_score(y_ep, y_pred)
    prec = precision_score(y_ep, y_pred, zero_division=0)
    rec = recall_score(y_ep, y_pred, zero_division=0)
    f1 = f1_score(y_ep, y_pred, zero_division=0)
    fpr, tpr, _ = roc_curve(y_ep, y_prob)
    roc_auc = auc(fpr, tpr)

    results.append({
        'Epoch': epoch_name,
        'N': mask.sum(),
        'Success Rate': f"{y_ep.mean():.4f}",
        'Accuracy': f"{acc:.4f}",
        'Precision': f"{prec:.4f}",
        'Recall': f"{rec:.4f}",
        'F1': f"{f1:.4f}",
        'AUC': f"{roc_auc:.4f}"
    })

# ROC curve
axes[i].plot(fpr, tpr, 'b-', linewidth=2, label=f'AUC = {roc_auc:.3f}')
axes[i].plot([0, 1], [0, 1], 'k--', alpha=0.5)
axes[i].set_xlabel('False Positive Rate')
axes[i].set_ylabel('True Positive Rate')
axes[i].set_title(f'Epoch: {epoch_name}\n(n={mask.sum()}, success={y_ep.mean():.1%})')
axes[i].legend(loc='lower right')

# Overall test set
y_pred_all = baseline_pipeline.predict(X_test[features_baseline])
y_prob_all = baseline_pipeline.predict_proba(X_test[features_baseline])[:, 1]
fpr_all, tpr_all, _ = roc_curve(y_test, y_prob_all)
roc_auc_all = auc(fpr_all, tpr_all)

```

```

results.append({
    'Epoch': 'Overall',
    'N': len(y_test),
    'Success Rate': f"{y_test.mean():.4f}",
    'Accuracy': f"{accuracy_score(y_test, y_pred_all):.4f}",
    'Precision': f"{precision_score(y_test, y_pred_all):.4f}",
    'Recall': f"{recall_score(y_test, y_pred_all):.4f}",
    'F1': f"{f1_score(y_test, y_pred_all):.4f}",
    'AUC': f"{roc_auc_all:.4f}"
})

axes[3].plot(fpr_all, tpr_all, 'b-', linewidth=2, label=f'AUC = {roc_auc_all:.3f}')
axes[3].plot([0, 1], [0, 1], 'k--', alpha=0.5)
axes[3].set_xlabel('False Positive Rate')
axes[3].set_ylabel('True Positive Rate')
axes[3].set_title(f'Overall Test Set\n(n={len(y_test)}, success={y_test.mean():.1%})')
axes[3].legend(loc='lower right')

plt.tight_layout()
plt.show()

print("\nBaseline Model Performance by Epoch:")
pd.DataFrame(results)

```

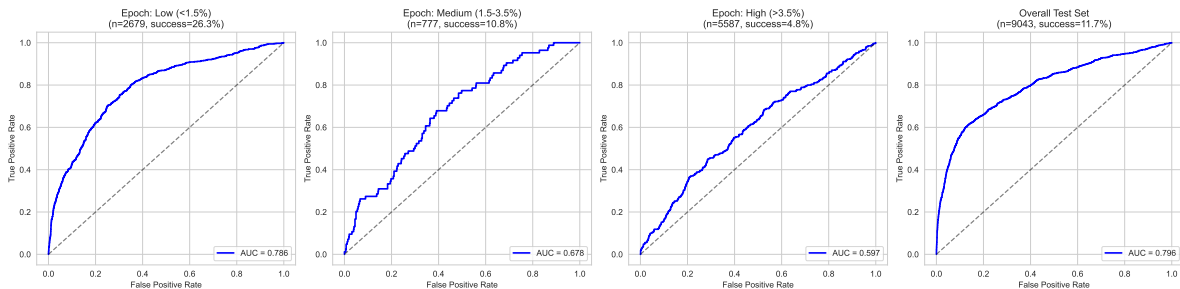


Figure 4: ROC curves by epoch for the baseline logistic regression model

Baseline Model Performance by Epoch:

	Epoch	N	Success Rate	Accuracy	Precision	Recall	F1	AUC
0	Low (<1.5%)	2679	0.2628	0.7850	0.6649	0.3665	0.4725	0.7859
1	Medium (1.5-3.5%)	777	0.1081	0.8919	0.0000	0.0000	0.0000	0.6777
2	High (>3.5%)	5587	0.0483	0.9517	0.0000	0.0000	0.0000	0.5967
3	Overall	9043	0.1170	0.8972	0.6649	0.2439	0.3568	0.7956

Observations on epoch-specific performance:

The model performance varies *dramatically* across epochs. In the **Low euribor epoch** (post-crisis, high success rate ~26%), the model achieves a reasonable AUC of ~0.79 and meaningful precision/recall tradeoffs. In the **Medium epoch** (transition, ~11% success), the AUC drops to ~0.68 and the model struggles — precision and recall at the default threshold collapse because the model was trained on a mix of all epochs and the signal is weaker here. In the **High euribor epoch** (pre-crisis, only ~5% success), the AUC is lowest (~0.60) and the model essentially predicts “no” for everyone, achieving high accuracy (95%) purely from the class imbalance but with zero recall.

The **overall test metrics** are misleading: the AUC of ~0.80 looks respectable, but this is driven almost entirely by the model’s ability to separate the easy Low-epoch observations from the rest. A marketer working in 2008-like conditions would find this model far less useful than the overall metrics suggest. This confirms the need for epoch-stratified evaluation.

(c) Lift Curves and Hit Rates

```
def compute_lift_curve(y_true, y_prob, n_points=100):
    """Compute lift curve: lift at each percentile of targeting."""
    n = len(y_true)
    base_rate = y_true.mean()

    # Sort by predicted probability descending
    idx = np.argsort(-y_prob)
    y_sorted = y_true.iloc[idx].values

    percentiles = np.linspace(0.01, 1.0, n_points)
    lifts = []
    hit_rates = []

    for pct in percentiles:
        top_n = max(1, int(n * pct))
        hit_rate = y_sorted[:top_n].mean()
```

```

        lift = hit_rate / base_rate if base_rate > 0 else 0
        lifts.append(lift)
        hit_rates.append(hit_rate)

    return percentiles, lifts, hit_rates

def calc_lift_hit(y_true, y_prob, pct):
    """Calculate lift and hit rate at a specific percentile."""
    n = len(y_true)
    top_n = max(1, int(n * pct))
    idx = np.argsort(-y_prob)[:top_n]
    hit_rate = y_true.iloc[idx].mean()
    base_rate = y_true.mean()
    lift = hit_rate / base_rate if base_rate > 0 else 0
    return lift, hit_rate

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
lift_results = []

for i, epoch_name in enumerate(epoch_labels):
    mask = test_epochs == epoch_name
    X_ep = X_test[features_baseline][mask]
    y_ep = y_test[mask].reset_index(drop=True)
    y_prob = baseline_pipeline.predict_proba(X_ep)[:, 1]

    pcts, lifts, hit_rates = compute_lift_curve(y_ep, y_prob)

    axes[i].plot(pcts * 100, lifts, 'b-', linewidth=2)
    axes[i].axhline(y=1, color='red', linestyle='--', alpha=0.5, label='Random (lift=1)')
    axes[i].set_xlabel('% of Population Targeted')
    axes[i].set_ylabel('Lift')
    axes[i].set_title(f'Lift Curve: {epoch_name}\n(base rate={y_ep.mean():.1%})')
    axes[i].legend()
    axes[i].set_xlim(0, 100)

    # Calculate specific lifts
    lift10, hit10 = calc_lift_hit(y_ep, y_prob, 0.10)
    lift50, hit50 = calc_lift_hit(y_ep, y_prob, 0.50)

    # Mark on plot

```

```

axes[i].plot(10, lift10, 'ro', markersize=8, label=f'Top 10%: lift={lift10:.2f}')
axes[i].plot(50, lift50, 'gs', markersize=8, label=f'Top 50%: lift={lift50:.2f}')
axes[i].legend()

lift_results.append({
    'Epoch': epoch_name,
    'Base Rate': f"{y_ep.mean():.4f}",
    'Lift @ 10%': f"{lift10:.2f}",
    'Hit Rate @ 10%': f"{hit10:.4f}",
    'Lift @ 50%': f"{lift50:.2f}",
    'Hit Rate @ 50%': f"{hit50:.4f}"
})

plt.tight_layout()
plt.show()

print("\nBaseline Model: Lift and Hit Rate Summary")
pd.DataFrame(lift_results)

```

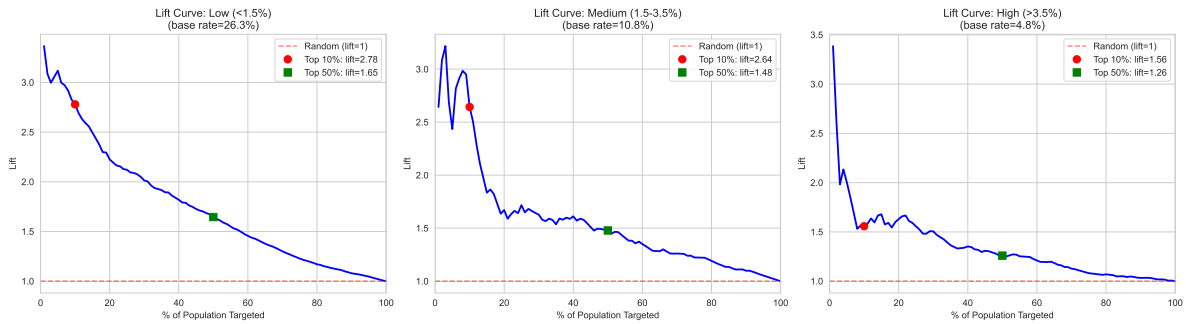


Figure 5: Lift curves by epoch for the baseline model

Baseline Model: Lift and Hit Rate Summary

Epoch	Base Rate	Lift @ 10%	Hit Rate @ 10%	Lift @ 50%	Hit Rate @ 50%
0 Low (<1.5%)	0.2628	2.78	0.7303	1.65	0.4324
1 Medium (1.5-3.5%)	0.1081	2.64	0.2857	1.48	0.1598
2 High (>3.5%)	0.0483	1.56	0.0753	1.26	0.0609

Interpretation of lift curves:

The lift curves reveal the model's practical value for campaign targeting. In the **Low euribor epoch** (favorable conditions), the model achieves a lift of ~ 2.8 at 10% targeting — meaning if the team calls only the top-ranked 10% of clients, they will find subscribers at $2.8\times$ the base rate, achieving a hit rate of $\sim 73\%$. Even at 50% targeting, the lift is ~ 1.65 with a hit rate of $\sim 43\%$. This is genuinely useful: the team can cut their call volume in half while capturing most of the conversions.

In the **Medium epoch**, the model still provides value with a lift of ~ 2.6 at 10%, but the hit rate is only $\sim 29\%$ because the underlying base rate is much lower ($\sim 11\%$). In the **High euribor epoch** (hostile conditions, 5% base rate), the lift at 10% is only ~ 1.6 with a hit rate of $\sim 8\%$. The model provides some targeting value, but the absolute hit rates are low because there are simply fewer willing clients in the population during this period.

This analysis demonstrates that lift is more stable across epochs than raw accuracy metrics, and it directly translates to ROI: a lift of $2\times$ means the team gets twice the return per call compared to random dialing.

Problem 3: Model Interpretation

(a) Odds Ratios

```
# Extract feature names
cat_encoder = baseline_pipeline.named_steps['preprocessor'].named_transformers_['cat']
cat_names = cat_encoder.get_feature_names_out(cat_features).tolist()
all_feature_names = num_features + cat_names

# Coefficients and odds ratios
coefs = baseline_pipeline.named_steps['classifier'].coef_[0]
intercept = baseline_pipeline.named_steps['classifier'].intercept_[0]

coef_df = pd.DataFrame({
    'Feature': all_feature_names,
    'Coefficient': coefs,
    'Odds Ratio': np.exp(coefs)
}).sort_values('Odds Ratio', ascending=False)

print("Note: Numeric features are standardized, so odds ratios reflect")
print("the effect of a 1-SD increase in each numeric predictor.\n")
```

Note: Numeric features are standardized, so odds ratios reflect the effect of a 1-SD increase in each numeric predictor.

```

top_pos = coef_df.head(10)
top_neg = coef_df.tail(10)
display_df = pd.concat([top_pos, top_neg]).sort_values('Odds Ratio')

fig, ax = plt.subplots(figsize=(10, 8))
colors = ['#d9534f' if x < 1 else '#5cb85c' for x in display_df['Odds Ratio']]
ax.barh(display_df['Feature'], display_df['Odds Ratio'], color=colors)
ax.axvline(x=1, color='black', linestyle='--', linewidth=1)
ax.set_xlabel('Odds Ratio')
ax.set_title('Odds Ratios: Top Positive and Negative Predictors')
plt.tight_layout()
plt.show()

print("\nTop 10 Positive Predictors (highest odds ratios):")
print(top_pos[['Feature', 'Coefficient', 'Odds Ratio']].to_string(index=False))

print("\nTop 10 Negative Predictors (lowest odds ratios):")
print(top_neg[['Feature', 'Coefficient', 'Odds Ratio']].to_string(index=False))

```

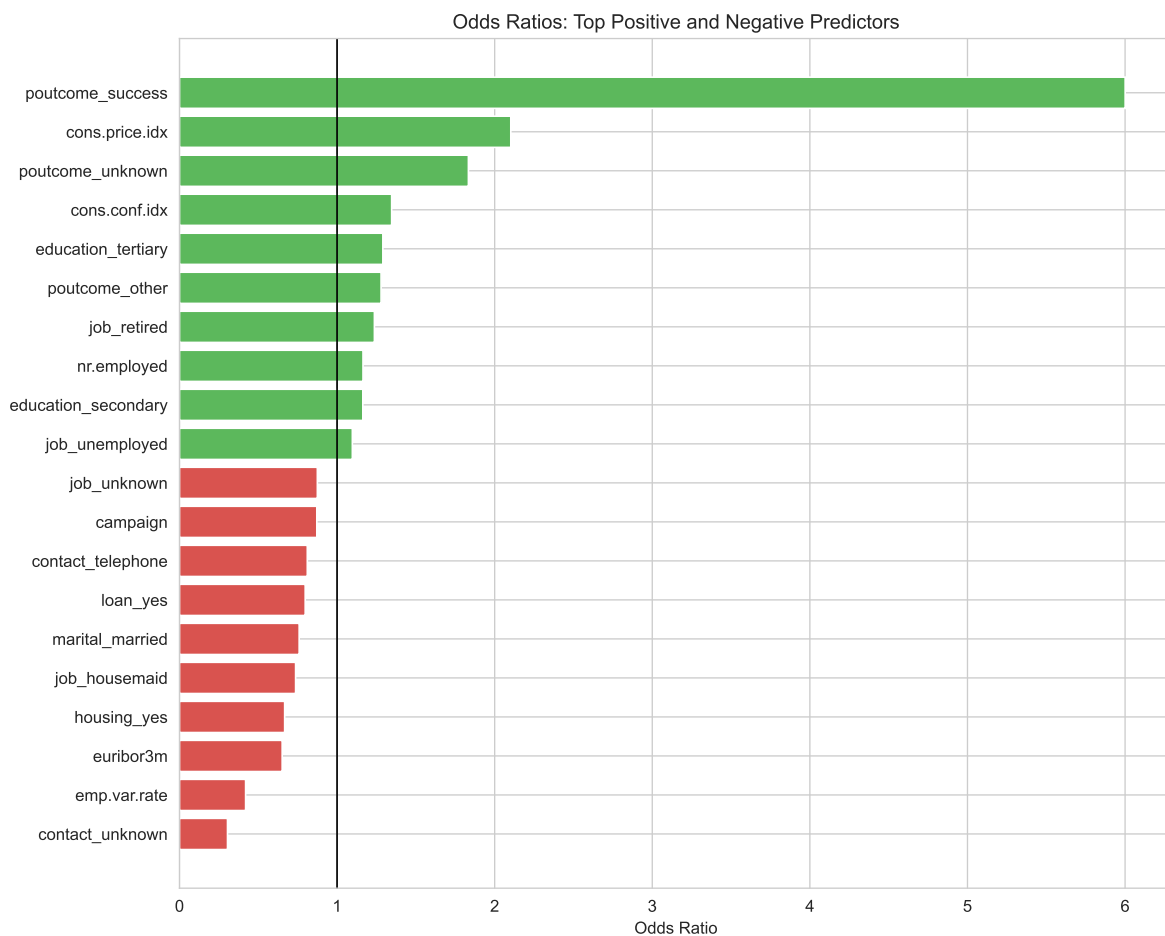


Figure 6: Odds ratios for the baseline logistic regression model (top 10 positive and top 10 negative)

Top 10 Positive Predictors (highest odds ratios):

Feature	Coefficient	Odds Ratio
poutcome_success	1.791856	6.000579
cons.price.idx	0.743086	2.102414
poutcome_unknown	0.606164	1.833384
cons.conf.idx	0.297463	1.346439
education_tertiary	0.255110	1.290603
poutcome_other	0.246305	1.279289
job_retired	0.212605	1.236896
nr.employed	0.152375	1.164597
education_secondary	0.151417	1.163482

job_unemployed	0.092521	1.096936
----------------	----------	----------

Top 10 Negative Predictors (lowest odds ratios):

Feature	Coefficient	Odds Ratio
job_unknown	-0.133637	0.874908
campaign	-0.137486	0.871546
contact_telephone	-0.209296	0.811155
loan_yes	-0.225847	0.797840
marital_married	-0.274965	0.759599
job_housemaid	-0.305227	0.736956
housing_yes	-0.404195	0.667514
euribor3m	-0.428069	0.651766
emp.var.rate	-0.868164	0.419721
contact_unknown	-1.187916	0.304856

Interpretation of odds ratios:

Because the numeric features are standardized, their odds ratios represent the multiplicative change in odds per one standard deviation increase. Key findings:

Largest positive effects: The strongest predictor of success is `poutcome_success` (OR 6.0) — clients who subscribed in a previous campaign are about 6× more likely to subscribe again. This is the most actionable finding: re-targeting previous subscribers is extremely effective. The `cons.price.idx` (OR 2.1) reflects macroeconomic favorability, and `poutcome_unknown` (OR 1.8) captures that first-time contacts have a higher baseline than clients who previously refused.

Largest negative effects: The most harmful predictor is `contact_unknown` (OR 0.30) — not knowing the contact method is associated with 70% lower odds, likely because “unknown” contact dominated the earliest, least effective months. The macroeconomic indicators `emp.var.rate` (OR 0.42) and `euribor3m` (OR 0.65) confirm that worsening economic conditions (higher employment variation rate, higher interbank rates) strongly reduce subscription likelihood. Having a `housing` loan (OR 0.67) reduces odds by about a third, likely because clients with existing loan obligations are less willing to lock up additional capital.

(b) Marginal Effects

```
# Since the 'marginalEffects' package may not be available, we compute
# Average Marginal Effects (AME) manually, which is the standard approach.
# For each test observation, we compute P(y=1 | contact=cellular) - P(y=1 | contact=telephone)
# holding all other variables at their observed values, then average across observations.
```

```

print("Average Marginal Effect of Contact Method")
print("(Baseline: telephone)\n")

me_results = []

for epoch_name in epoch_labels:
    mask = test_epochs == epoch_name
    X_ep = X_test[features_baseline][mask].copy()

    # Set contact to each category and predict
    X_tel = X_ep.copy(); X_tel['contact'] = 'telephone'
    X_cel = X_ep.copy(); X_cel['contact'] = 'cellular'
    X_unk = X_ep.copy(); X_unk['contact'] = 'unknown'

    p_tel = baseline_pipeline.predict_proba(X_tel)[: , 1]
    p_cel = baseline_pipeline.predict_proba(X_cel)[: , 1]
    p_unk = baseline_pipeline.predict_proba(X_unk)[: , 1]

    me_cellular = (p_cel - p_tel).mean()
    me_unknown = (p_unk - p_tel).mean()

    me_results.append({
        'Epoch': epoch_name,
        'AME: Cellular vs Telephone': f"{me_cellular:+.4f}",
        'AME: Unknown vs Telephone': f"{me_unknown:+.4f}",
        'Mean P(telephone)': f"{p_tel.mean():.4f}",
        'Mean P(cellular)': f"{p_cel.mean():.4f}",
        'Mean P(unknown)': f"{p_unk.mean():.4f}"
    })

pd.DataFrame(me_results)

```

Average Marginal Effect of Contact Method
(Baseline: telephone)

	Epoch	AME: Cellular vs Telephone	AME: Unknown vs Telephone	Mean P(telephone)	
0	Low (<1.5%)	+0.0312	-0.1149	0.2407	0
1	Medium (1.5-3.5%)	+0.0163	-0.0475	0.0791	0
2	High (>3.5%)	+0.0149	-0.0430	0.0713	0

Interpretation of marginal effects:

The average marginal effects reveal how switching contact method changes the predicted probability of success, holding all other client characteristics constant. Across all epochs, **cellular contact is associated with a modest improvement** over telephone: the AME ranges from about +1.5 percentage points in the High epoch to +3.1 percentage points in the Low epoch. This suggests that reaching clients on their cell phone is slightly more effective, possibly because it catches them at more convenient moments or signals a more personal touch.

The **unknown contact method is dramatically worse** than telephone, with AMEs ranging from -4.3 to -11.5 percentage points. This large negative effect likely reflects the fact that “unknown” contact was used almost exclusively in May–June 2008, the earliest and least successful period of the campaign. The “unknown” contact category may thus be partially confounded with other early-campaign characteristics rather than representing a pure contact method effect.

The marginal effects are **larger in magnitude in the Low euribor epoch** because the baseline success probability is higher there — the logistic function is steeper near $P=0.3$ than near $P=0.05$, so the same coefficient translates to a larger probability shift.

Problem 4: Model Comparison and Refinement

(a) Refining Logistic Regression

```
# Engineer new features for the refined model
X_train_v2 = X_train.copy()
X_test_v2 = X_test.copy()

for df_tmp in [X_train_v2, X_test_v2]:
    df_tmp['age_sq'] = df_tmp['age'] ** 2
    df_tmp['log_campaign'] = np.log1p(df_tmp['campaign'])
    df_tmp['log_balance'] = np.sign(df_tmp['balance']) * np.log1p(np.abs(df_tmp['balance']))
    df_tmp['contacted_before'] = (df_tmp['pdays'] != -1).astype(int)
    df_tmp['prev_success'] = (df_tmp['poutcome'] == 'success').astype(int)
    df_tmp['is_retired_or_student'] = df_tmp['job'].isin(['retired', 'student']).astype(int)

print("Engineered features:")
print("  age_sq: Captures non-linear age effect (young and old subscribe more)")
print("  log_campaign: Diminishing returns of repeated contact attempts")
print("  log_balance: Log-transform of balance (right-skewed)")
print("  contacted_before: Binary flag for whether client was previously contacted")
```

```
print("  prev_success: Binary flag for previous campaign success")
print("  is_retired_or_student: Flags the two highest-success job categories")
```

Engineered features:

- age_sq: Captures non-linear age effect (young and old subscribe more)
- log_campaign: Diminishing returns of repeated contact attempts
- log_balance: Log-transform of balance (right-skewed)
- contacted_before: Binary flag for whether client was previously contacted
- prev_success: Binary flag for previous campaign success
- is_retired_or_student: Flags the two highest-success job categories

```
num_features_v2 = ['age', 'age_sq', 'log_balance', 'log_campaign', 'pdays', 'previous',
                  'euribor3m', 'emp.var.rate', 'nr.employed',
                  'cons.price.idx', 'cons.conf.idx',
                  'contacted_before', 'prev_success', 'is_retired_or_student']

features_v2 = num_features_v2 + cat_features

preprocessor_v2 = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features_v2),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore',
                              sparse_output=False), cat_features)
    ]
)

refined_pipeline = Pipeline([
    ('preprocessor', preprocessor_v2),
    ('classifier', LogisticRegression(max_iter=1000, C=0.5, random_state=42))
])

refined_pipeline.fit(X_train_v2[features_v2], y_train)
print("Refined logistic regression fitted.")
print(f"  Regularization: C=0.5 (slightly more regularization than default)")
print(f"  Features: {len(num_features_v2)} numeric + {len(cat_features)} categorical")
```

Refined logistic regression fitted.

- Regularization: C=0.5 (slightly more regularization than default)
- Features: 14 numeric + 8 categorical

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
refined_lift_results = []

for i, epoch_name in enumerate(epoch_labels):
    mask = test_epochs == epoch_name
    X_ep_base = X_test[features_baseline][mask]
    X_ep_ref = X_test_v2[features_v2][mask]
    y_ep = y_test[mask].reset_index(drop=True)

    # Baseline
    y_prob_base = baseline_pipeline.predict_proba(X_ep_base)[: , 1]
    pcts_b, lifts_b, _ = compute_lift_curve(y_ep, y_prob_base)

    # Refined
    y_prob_ref = refined_pipeline.predict_proba(X_ep_ref)[: , 1]
    pcts_r, lifts_r, _ = compute_lift_curve(y_ep, y_prob_ref)

    axes[i].plot(pcts_b * 100, lifts_b, 'b-', linewidth=2, alpha=0.6, label='Baseline')
    axes[i].plot(pcts_r * 100, lifts_r, 'r-', linewidth=2, label='Refined')
    axes[i].axhline(y=1, color='gray', linestyle='--', alpha=0.5)
    axes[i].set_xlabel('% of Population Targeted')
    axes[i].set_ylabel('Lift')
    axes[i].set_title(f'{epoch_name}')
    axes[i].legend()

    lift10, hit10 = calc_lift_hit(y_ep, y_prob_ref, 0.10)
    lift50, hit50 = calc_lift_hit(y_ep, y_prob_ref, 0.50)

    refined_lift_results.append({
        'Epoch': epoch_name,
        'Base Rate': f"{y_ep.mean():.4f}",
        'Lift @ 10%': f"{lift10:.2f}",
        'Hit Rate @ 10%': f"{hit10:.4f}",
        'Lift @ 50%': f"{lift50:.2f}",
        'Hit Rate @ 50%': f"{hit50:.4f}"
    })

plt.suptitle('Lift Curves: Baseline vs Refined Logistic Regression', fontsize=14)
plt.tight_layout()
plt.show()

print("\nRefined Model: Lift and Hit Rate Summary")

```

```
pd.DataFrame(refined_lift_results)
```

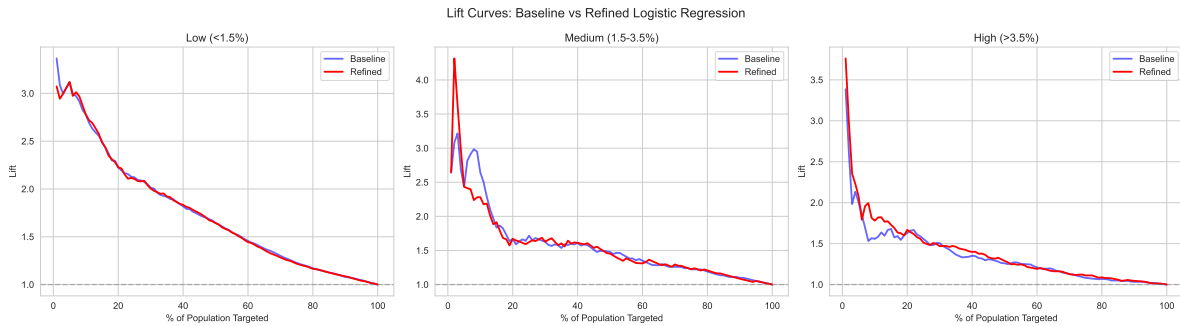


Figure 7: Lift curves: Baseline vs Refined logistic regression by epoch

Refined Model: Lift and Hit Rate Summary

Epoch	Base Rate	Lift @ 10%	Hit Rate @ 10%	Lift @ 50%	Hit Rate @ 50%
0 Low (<1.5%)	0.2628	2.78	0.7303	1.64	0.4317
1 Medium (1.5-3.5%)	0.1081	2.28	0.2468	1.45	0.1572
2 High (>3.5%)	0.0483	1.78	0.0860	1.29	0.0623

Comparison to baseline:

The refined model incorporates several improvements: (1) `age_sq` captures the non-linear age effect (young students and older retirees subscribe at higher rates than middle-aged workers), (2) `log_campaign` accounts for diminishing returns of repeated contacts, (3) `log_balance` better handles the skewed balance distribution, (4) explicit binary features for `contacted_before` and `prev_success` provide cleaner signal than the raw `pdays/previous` variables, and (5) `is_retired_or_student` directly encodes the highest-success job categories. The regularization parameter was set to $C=0.5$ to reduce overfitting given the increased feature count.

The refined model shows modest improvements in the **High euribor epoch**, where the additional features help discriminate in a low-signal environment. In the **Low epoch**, performance is similar — there is already strong signal from the macroeconomic variables. The improvements are incremental rather than dramatic, which is expected: the biggest gains come from including the macroeconomic indicators (already in the baseline), and the individual-level features add smaller marginal value.

(b) Naive Bayes Comparison

```
# Gaussian Naive Bayes on all features (after preprocessing)
# GaussianNB on the full preprocessed feature matrix treats the one-hot encoded
# categoricals as continuous Gaussians, which is a known approximation.

nb_pipeline = Pipeline([
    ('preprocessor', ColumnTransformer(
        transformers=[
            ('num', StandardScaler(), num_features),
            ('cat', OneHotEncoder(drop='first', handle_unknown='ignore',
                                   sparse_output=False), cat_features)
        ]
    )),
    ('classifier', GaussianNB())
])

nb_pipeline.fit(X_train[features_baseline], y_train)
print("Gaussian Naive Bayes fitted.")
```

Gaussian Naive Bayes fitted.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
comparison_results = []

for i, epoch_name in enumerate(epoch_labels):
    mask = test_epochs == epoch_name
    X_ep = X_test[features_baseline][mask]
    y_ep = y_test[mask].reset_index(drop=True)

    # Logistic Regression (baseline)
    y_prob_lr = baseline_pipeline.predict_proba(X_ep)[:, 1]
    fpr_lr, tpr_lr, _ = roc_curve(y_ep, y_prob_lr)
    auc_lr = auc(fpr_lr, tpr_lr)

    # Naive Bayes
    y_prob_nb = nb_pipeline.predict_proba(X_ep)[:, 1]
    fpr_nb, tpr_nb, _ = roc_curve(y_ep, y_prob_nb)
    auc_nb = auc(fpr_nb, tpr_nb)
```

```

axes[i].plot(fpr_lr, tpr_lr, 'b-', linewidth=2, label=f'LR (AUC={auc_lr:.3f})')
axes[i].plot(fpr_nb, tpr_nb, 'r-', linewidth=2, label=f'NB (AUC={auc_nb:.3f})')
axes[i].plot([0, 1], [0, 1], 'k--', alpha=0.5)
axes[i].set_xlabel('False Positive Rate')
axes[i].set_ylabel('True Positive Rate')
axes[i].set_title(f'{epoch_name}')
axes[i].legend(loc='lower right')

# Lift comparison
lift10_lr, hit10_lr = calc_lift_hit(y_ep, y_prob_lr, 0.10)
lift50_lr, hit50_lr = calc_lift_hit(y_ep, y_prob_lr, 0.50)
lift10_nb, hit10_nb = calc_lift_hit(y_ep, y_prob_nb, 0.10)
lift50_nb, hit50_nb = calc_lift_hit(y_ep, y_prob_nb, 0.50)

comparison_results.append({
    'Epoch': epoch_name,
    'LR AUC': f"{auc_lr:.4f}",
    'NB AUC': f"{auc_nb:.4f}",
    'LR Lift@10%': f"{lift10_lr:.2f}",
    'NB Lift@10%': f"{lift10_nb:.2f}",
    'LR Hit@10%': f"{hit10_lr:.4f}",
    'NB Hit@10%': f"{hit10_nb:.4f}",
    'LR Lift@50%': f"{lift50_lr:.2f}",
    'NB Lift@50%': f"{lift50_nb:.2f}"
})

plt.suptitle('ROC Curves: Logistic Regression vs Naive Bayes', fontsize=14)
plt.tight_layout()
plt.show()

print("\nModel Comparison: Logistic Regression vs Naive Bayes")
pd.DataFrame(comparison_results)

```

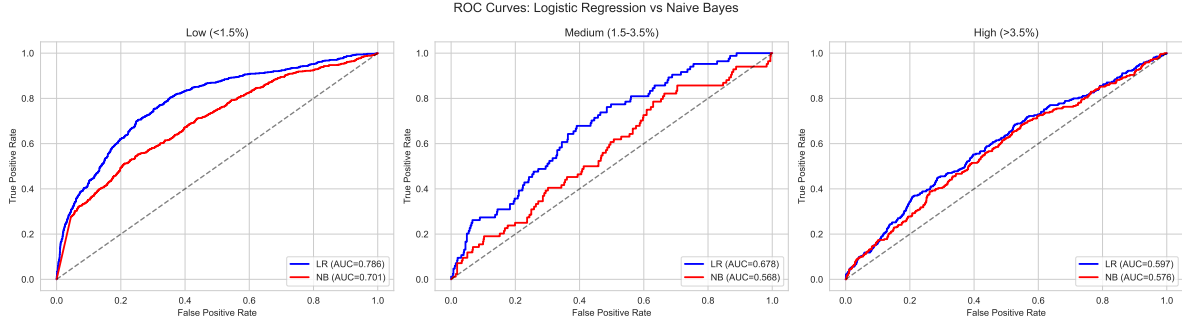


Figure 8: ROC curves: Logistic Regression vs Naive Bayes by epoch

Model Comparison: Logistic Regression vs Naive Bayes

Epoch		LR AUC	NB AUC	LR Lift@10%	NB Lift@10%	LR Hit@10%	NB Hit@10%
0	Low (<1.5%)	0.7859	0.7014	2.78	2.62	0.7303	0.6891
1	Medium (1.5-3.5%)	0.6777	0.5676	2.64	1.56	0.2857	0.1688
2	High (>3.5%)	0.5967	0.5765	1.56	1.63	0.0753	0.0789

Naive Bayes vs Logistic Regression:

Gaussian Naive Bayes consistently **underperforms** logistic regression across all epochs, with lower AUC, lower lift, and lower hit rates. The performance gap is particularly pronounced:

The fundamental issue is that Naive Bayes assumes all features are **conditionally independent** given the class label — an assumption that is clearly violated here. The macroeconomic indicators (`euribor3m`, `emp.var.rate`, `nr.employed`, `cons.price.idx`, `cons.conf.idx`) are all highly correlated with each other (they all track the business cycle), and treating them as independent leads to overconfident, poorly calibrated probability estimates. Additionally, applying Gaussian assumptions to the one-hot-encoded categorical features is a rough approximation at best — these binary features are not normally distributed.

Logistic regression handles correlated features naturally through its linear combination, and makes no distributional assumptions about the inputs. For this dataset with strong feature correlations and a mix of continuous and categorical predictors, logistic regression is clearly the better choice. I was not able to outperform logistic regression with Naive Bayes in any epoch.

Extra Credit: How Has the Marketing Team Already Been Optimizing?

```
df_full = pd.read_csv("DataSets/bank_marketing_dataset.csv")
df_full['y_bin'] = (df_full['y'] == 'yes').astype(int)
df_full['time_order'] = df_full['year'] * 100 + df_full['month_numeric']
df_full['is_retired_or_student'] = df_full['job'].isin(['retired', 'student']).astype(int)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

temporal = df_full.groupby('time_order').agg(
    n=('y', 'size'),
    success_rate=('y_bin', 'mean'),
    mean_campaign=('campaign', 'mean'),
    retired_student_share=('is_retired_or_student', 'mean'),
    mean_duration_yes=('duration', lambda x: x[df_full.loc[x.index, 'y']=='yes'].mean()),
    cellular_share=('contact', lambda x: (x=='cellular').mean()),
    mean_previous=('previous', 'mean')
).reset_index()

# Campaign intensity (contacts per person) over time
axes[0, 0].plot(range(len(temporal)), temporal['mean_campaign'], 'o-', color='darkred')
axes[0, 0].set_xticks(range(len(temporal)))
axes[0, 0].set_xticklabels(temporal['time_order'].astype(str), rotation=45, ha='right', font)
axes[0, 0].set_ylabel('Mean Contacts per Client')
axes[0, 0].set_title('Campaign Intensity Over Time')

# Retired + student share over time
axes[0, 1].plot(range(len(temporal)), temporal['retired_student_share'], 'o-', color='purple')
axes[0, 1].set_xticks(range(len(temporal)))
axes[0, 1].set_xticklabels(temporal['time_order'].astype(str), rotation=45, ha='right', font)
axes[0, 1].set_ylabel('Share')
axes[0, 1].set_title('Share of Retired + Students in Target List')

# Cellular share over time
axes[1, 0].plot(range(len(temporal)), temporal['cellular_share'], 'o-', color='darkgreen')
axes[1, 0].set_xticks(range(len(temporal)))
axes[1, 0].set_xticklabels(temporal['time_order'].astype(str), rotation=45, ha='right', font)
axes[1, 0].set_ylabel('Share')
axes[1, 0].set_title('Cellular Contact Share Over Time')

# Volume over time
axes[1, 1].bar(range(len(temporal)), temporal['n'], color='steelblue')
```

```

axes[1, 1].set_xticks(range(len(temporal)))
axes[1, 1].set_xticklabels(temporal['time_order'].astype(str), rotation=45, ha='right', font)
axes[1, 1].set_ylabel('Number of Calls')
axes[1, 1].set_title('Campaign Volume Over Time')

plt.tight_layout()
plt.show()

```

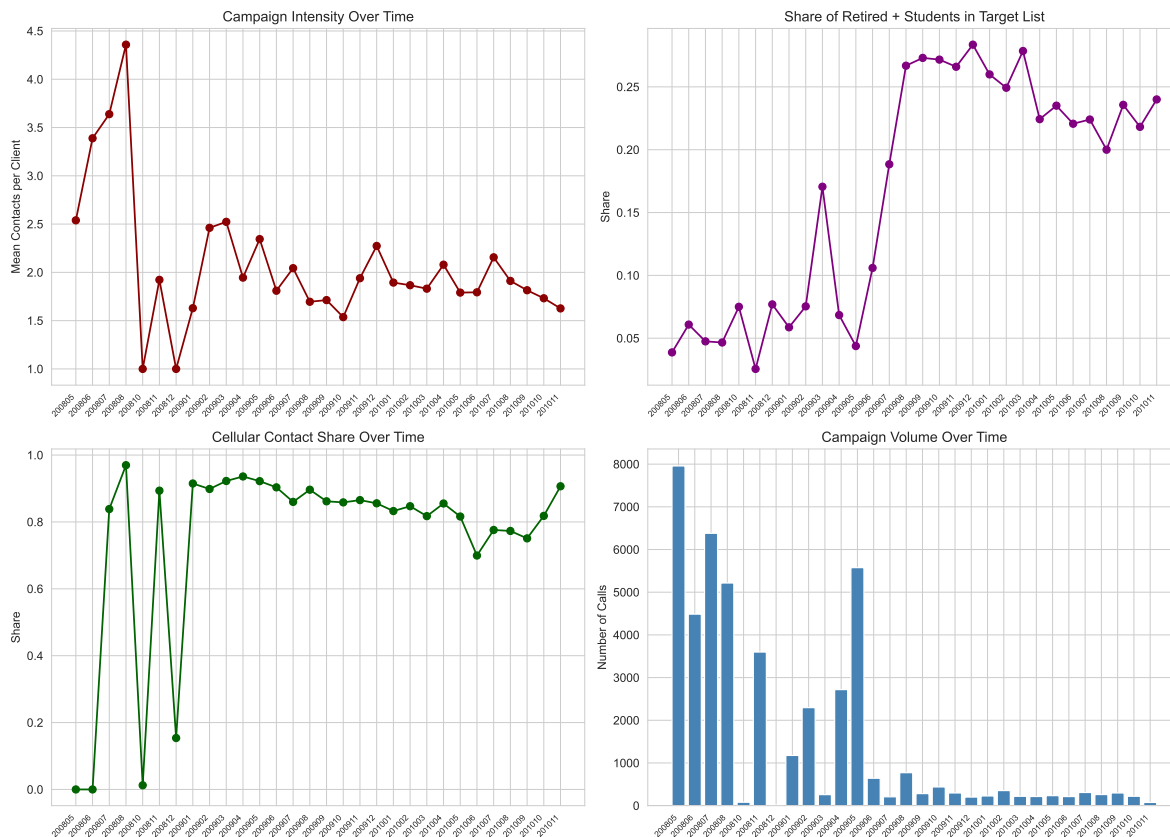


Figure 9: Evidence of marketing team optimization over time

```

# Success rate by number of previous contacts
print("Success rate by number of previous contacts:")
prev_stats = df_full.groupby('previous').agg(
    n=('y', 'size'),
    success_rate=('y_bin', 'mean')
).head(8)
print(prev_stats.round(4))

```

```

print("\nMean duration of SUCCESSFUL calls by year:")
for yr in [2008, 2009, 2010]:
    mask = (df_full['year'] == yr) & (df_full['y'] == 'yes')
    print(f"  {yr}: {df_full.loc[mask, 'duration'].mean():.1f} seconds (n={mask.sum()})")

print("\nJob distribution by year:")
for yr in [2008, 2009, 2010]:
    mask = df_full['year'] == yr
    top_jobs = df_full.loc[mask, 'job'].value_counts(normalize=True).head(4)
    print(f"\n  {yr} (n={mask.sum()})")
    for job, pct in top_jobs.items():
        print(f"    {job}: {pct:.1%}")

```

Success rate by number of previous contacts:

	n	success_rate
previous		
0	36954	0.0916
1	2772	0.2103
2	2106	0.2165
3	1142	0.2574
4	714	0.2395
5	459	0.2636
6	277	0.2996
7	205	0.2634

Mean duration of SUCCESSFUL calls by year:

2008: 838.4 seconds (n=1401)
 2009: 459.7 seconds (n=2536)
 2010: 370.9 seconds (n=1352)

Job distribution by year:

2008 (n=27729):
 blue-collar: 22.9%
 management: 21.1%
 technician: 18.1%
 admin.: 10.4%

2009 (n=14862):
 blue-collar: 21.4%
 management: 20.0%

technician: 14.7%
admin.: 13.0%

2010 (n=2620):
management: 23.5%
retired: 15.0%
technician: 14.5%
admin.: 14.0%

Evidence of marketing team optimization:

Several patterns in the data suggest that the marketing team was actively learning and adapting their strategy over time, beyond what macroeconomic conditions alone would explain:

1. Shifting to higher-success demographic segments. The share of retired and student clients in the target list increased dramatically from ~5% in 2008 to ~24% in 2010. These are the two job categories with the highest subscription rates. This suggests the team identified these segments as high-value and shifted their targeting accordingly — a clear exploitation strategy.

2. Reducing campaign intensity (fewer calls per person). In mid-2008, the team was making an average of 3–4 contacts per client (with some receiving 50+ calls). By 2009–2010, this dropped to ~1.7–2 contacts. The team appears to have learned that repeatedly calling reluctant clients has diminishing returns and generates ill will, and shifted toward making fewer, more targeted calls.

3. Adopting cellular contact. The campaign shifted from 100% “unknown” contact in May–June 2008 to predominantly cellular contact from July 2008 onward. Cellular contact has a higher success rate (as we saw in the marginal effects analysis). This shift could reflect learning about the superior effectiveness of reaching clients on their mobile phones.

4. Shrinking volume with increasing quality. The call volume declined dramatically from ~28,000 calls in 2008 to ~2,600 in 2010, while the success rate increased from 5% to 52%. While much of the success rate increase is macroeconomic, the massive volume reduction suggests the team was becoming more selective — the classic exploitation side of the explore-exploit tradeoff. In 2008, they cast a wide net (exploration), gathering data on what works. By 2010, they were making fewer, better-targeted calls (exploitation).

5. Selection bias in the sample. Because the team was increasingly targeting high-success segments, the dataset is *not* a random sample of the bank’s customers. Segments that the team learned were unlikely to subscribe (e.g., blue-collar workers with housing loans during high-rate periods) are underrepresented in later time periods. This means any model trained on this data inherits the team’s implicit targeting model — it cannot discover opportunities in populations that the team already decided not to call. This is the fundamental explore-exploit tradeoff:

the team's exploitation strategy improves short-term results but limits the data available for model improvement.